Switzerland's Export Structure

Annual merchandise exports by HS chapter, 2022–2025

Table of contents

1 Summary and current scope **1**

2 Requirements and configuration **2**
- 2.1 Reproduction options . . . 2
- 2.2 API key . . . 3
- 2.3 Swiss export query settings . . . 3
- 2.4 Readable query settings . . . 5

3 ELT process **5**
- 3.1 Extract: request a single year . . . 5
- 3.2 Extract: collect the selected years . . . 6
- 3.3 Inspect and validate the raw extraction . . . 7
- 3.4 Save the validated raw-data snapshot . . . 9
- 3.5 Readable trade-data CSV . . . 11
- 3.6 Extraction interpretation and next steps . . . 11

4 Sources and method references **11**

5 PDF export and final submission checklist **12**

1 Summary and current scope

This project examines **Switzerland's export structure from 2022 to 2025** using UN Comtrade merchandise trade statistics. This version implements extraction, input validation, and snapshot saving; MongoDB modeling and analysis follow next.

The scope is annual exports to the **World** at the **two-digit HS chapter** level. Switzerland's Comtrade reporter code is **757**. We request the original HS classification and require all returned records to use **H6 (HS 2022)** for comparability.

Research questions for the later MongoDB analysis:

1. Which chapters have the highest export value in 2025?
2. Which chapters gained or lost export share between 2022 and 2025?
3. Which chapters had the largest absolute and percentage changes in export value?
4. How concentrated were exports in the top five chapters each year?

Shares will initially use the sum of the included chapter values, not an independently verified national total. Values are in current US dollars; changes are not inflation-adjusted or direct measures of physical trade volumes. This is country-level trade, not company sales or profit.

Current workflow: configure → request JSON → collect raw records → inspect and validate.

Later: load into MongoDB → clean and model inside MongoDB → analyse with aggregation pipelines → add diagrams and export the report. No database writes occur in this version.

2 Requirements and configuration

Use a Python kernel with `requests`, `pandas`, and `python-dotenv` installed. Install the repository-local dependencies with `python -m pip install -r requirements.txt` from this notebook's folder, using the same Python environment as the notebook kernel. `python-dotenv` loads a local `.env` file; the other imports belong to Python's standard library.

The course template uses `requests.get()`, `.json()`, a list of records, and a loop with a delay. I will keep that approach. Authentication, query parameters, and response fields follow the UN Comtrade documentation. Timeouts and bounded rate-limit retries are small reliability additions.

```
import os
import time
import json
import math
from datetime import datetime, timezone
from getpass import getpass
from pathlib import Path
from pprint import pprint

import pandas as pd
import requests
from dotenv import load_dotenv

# Keep notebook tables compact enough for reading and later PDF export.
pd.set_option("display.max_rows", 20)
pd.set_option("display.max_colwidth", 60)
```

2.1 Reproduction options

Option A — fresh API extraction: leave `DATA_SOURCE = "api"`. The notebook downloads data through the API and later saves a dated raw-data snapshot after validation. Source revisions can change results between downloads.

Option B — saved snapshot data: set `DATA_SOURCE = "snapshot"` and set `SNAPSHOT_PATH` to the JSON file supplied in the project's `data` folder. Restart the kernel and run from the top. No API key or network request is needed for this mode; it reads the same raw records and query log used for the saved results. This option supports reproducibility and does not replace the assignment's demonstrated API extraction.

The path below is a placeholder until the first Swiss snapshot is created. Use the dated filename printed by the saving cell. Both modes must match the current query settings and pass the same record validation. The earlier US pilot snapshots do not match this scope.

```
DATA_SOURCE = "api"
SNAPSHOT_PATH = Path("data/trade_snapshot.json")

if DATA_SOURCE not in {"api", "snapshot"}:
    raise ValueError("Choose either api or snapshot as DATA_SOURCE.")
```

2.2 API key

Only needed for fresh extraction; skipped in snapshot mode. Use your own free key:

1. Follow UN Comtrade's free account registration instructions.
2. Open My API Portal, generate a key if needed, and copy either the primary or secondary key. Free Basic Individual access is sufficient for this scope.
3. Set `DATA_SOURCE = "api"`. Copy `.env.example` to `.env` beside the notebook and fill in `UN_COMTRADE_API_KEY=` with your key, or use the hidden prompt.
4. Restart the kernel and run from the top with this notebook's folder as the working directory. Existing environment values take precedence over `.env`.

Reference: the official example notebook uses the `comtradeapicall` helper package. Here we use `requests` directly to follow the course; no helper package is required.

To reproduce without a key, choose **snapshot mode** and the supplied dated JSON file. Do not submit `.env` or show keys in code or outputs.

```
headers = {}
# Read the secret without writing it into the notebook source.
if DATA_SOURCE == "api":
    load_dotenv(dotenv_path=".env", override=False)
    api_key = os.environ.get("UN_COMTRADE_API_KEY", "").strip()
    if not api_key:
        api_key = getpass("UN Comtrade API key: ").strip()

    if not api_key:
        raise ValueError("An API key is required for this authenticated endpoint.")

    # A header keeps the key out of the query URL.
    headers = {"Ocp-Apim-Subscription-Key": api_key}
    print("API key loaded. Its value is not displayed.")
else:
    print("Snapshot mode: no API key is loaded or requested.")
```

```
API key loaded. Its value is not displayed.
```

2.3 Swiss export query settings

Setting	Meaning
`C / A / HS` in the endpoint	Goods, annual frequency, original HS classification
`reporterCode = 757`	Switzerland; use the Comtrade code, not ISO numeric code 756
`partnerCode = 0`	World total, not a particular destination
`cmdCode = "AG2"`	Request all two-digit HS chapters; individual returned codes remain strings
`flowCode = "X"`	Exports
`partner2Code = 0`, `customsCode = "C00"`, `motCode = 0`	Aggregate selections for the additional dimensions
`includeDesc = "true"`	Request readable descriptions alongside codes
`maxRecords = 500`	Conservative request cap, not a guarantee of completeness

We request 2022, 2023, 2024, and 2025 separately. Public API checks returned 96 chapters per year, all in H6. We explicitly expect chapters 01–97 excluding 77, which is reserved; 98/99 special categories are outside this analytical scope. The notebook stops if expected chapters are missing rather than assuming missing trade is zero. If a different edition is returned, resolve classification comparability before continuing. Although monthly 2026 data is available for some periods, it is excluded to keep complete calendar years comparable.

2.3.1 API parameter reference

Adapted from the UN Comtrade package's Selection Criteria. The package lists these arguments as strings; our direct HTTP requests can also supply numeric codes as Python integers through `params`.

Parameter	Meaning	Value in this project
`typeCode`	Product type: goods (`C`) or services (`S`)	`C` in the URL
`freqCode`	Annual (`A`) or monthly (`M`) frequency	`A` in the URL
`clCode`	Commodity classification and edition	`HS` in the URL; returned edition must be `H6`
`period`	Year (`YYYY`) or year-month (`YYYYMM`)	One year per request, 2022–2025
`reporterCode`	Country or area reporting the trade	`757`: Switzerland
`cmdCode`	Commodity selection within the classification	`AG2`: all two-digit chapters
`flowCode`	Trade flow, such as exports or imports	`X`: exports
`partnerCode`	Primary trading partner	`0`: World
`partner2Code`	Additional partner dimension, where reported	`0`: World/aggregate selection
`customsCode`	Customs or statistical procedure	`C00`: total
`motCode`	Mode of transport	`0`: total

`typeCode`, `freqCode`, and `clCode` fill the URL path `/C/A/HS`; the remaining selections are query parameters. `AG2` is a request selector, not a returned commodity code: records contain chapter codes such as `"01"` and `"85"`.

```
API_URL = "https://comtradeapi.un.org/data/v1/get/C/A/HS"
YEARS = ["2022", "2023", "2024", "2025"]
MAX_RECORDS = 500
REQUEST_DELAY = 5
EXPECTED_EDITION = "H6"
CHAPTER_CODES = {f"{number:02d}" for number in range(1, 98) if number != 77}

# These filters describe one aggregate observation per year and chapter.
base_params = {
    "reporterCode": 757,
    "partnerCode": 0,
    "partner2Code": 0,
    "cmdCode": "AG2",
    "flowCode": "X",
    "customsCode": "C00",
    "motCode": 0,
    "maxRecords": MAX_RECORDS,
    "includeDesc": "true"
}
```

2.4 Readable query settings

Export one row of query parameters per year to CSV for easy inspection. These are settings, not trade observations; no API key is included. A timestamp gives each export a new filename.

```
Path("data").mkdir(exist_ok=True)
query_csv = Path("data") / f"swiss_query_settings_{datetime.now(timezone.utc):%Y%m%dT%H%M%S%
pd.DataFrame([{**base_params, "period": year} for year in YEARS]).to_csv(query_csv, index=Fa
print(f"Query settings saved to {query_csv}")
```

```
Query settings saved to data/swiss_query_settings_20261015T180956597021Z.csv
```

3 ELT process

3.1 Extract: request a single year

`requests.get()` downloads the response. `.raise_for_status()` stops on an HTTP error, and `.json()` converts JSON into Python dictionaries and lists. Comtrade's actual trade records are in `payload["data"]`, rather than the cocktail template's `data["drinks"]`.

The function retries HTTP **429** (rate limit) at most twice after the first attempt. Other HTTP failures stop immediately. For **401/403**, check the key and account access rather than repeatedly retrying. A response reaching the record cap is rejected until the query is narrowed or a documented larger extraction strategy is chosen.

```
def extract_year(year):
    params = base_params.copy()
    params["period"] = year

    for attempt in range(3):
        response = requests.get(
            API_URL, headers=headers, params=params, timeout=30
        )

        # Pause and retry only when the service reports a rate limit.
        if response.status_code == 429 and attempt < 2:
            wait_seconds = 15 * (attempt + 1)
            print(f"Rate limit for {year}; retrying in {wait_seconds} seconds.")
            time.sleep(wait_seconds)
            continue

        response.raise_for_status()
        payload = response.json()

        if not isinstance(payload, dict):
            raise ValueError("Expected a JSON object from Comtrade.")
        if payload.get("error"):
            raise ValueError("Comtrade reported an API error; review the query and documenta
        if not isinstance(payload.get("data"), list):
            raise ValueError("Expected a list of records in the API's data field.")

        records = payload["data"]
        if len(records) >= MAX_RECORDS:
            raise ValueError("The record cap was reached; completeness is not established.")

        return records
```

3.2 Extract: collect the selected years

Like the course template, we loop over query values, add the returned documents to a list, and pause between requests. Here the query values are years, not the first letters of drink names.

`list_trade` preserves the returned trade fields. `extraction_log` separately records when each query was made and how many records it returned. In snapshot mode, these are restored from the supplied JSON file instead. This cell does not write to MongoDB or save a new file. Snapshot query metadata must match the current settings.

```
list_trade = []
extraction_log = []
query_scope = {
    "endpoint": API_URL, "years": YEARS, "filters": base_params,
    "expected_edition": EXPECTED_EDITION, "chapters": sorted(CHAPTER_CODES)
}

if DATA_SOURCE == "snapshot":
    with SNAPSHOT_PATH.open(encoding="utf-8") as file:
```

```
        snapshot = json.load(file)
    if not isinstance(snapshot, dict) or snapshot.get("schema_version") != 1:
        raise ValueError("Unsupported snapshot format.")
    if snapshot.get("query_scope") != query_scope:
        raise ValueError("Snapshot query settings differ from this notebook's settings.")
    if not isinstance(snapshot.get("records"), list):
        raise ValueError("Snapshot must contain a list of raw records.")
    if not isinstance(snapshot.get("extraction_log"), list):
        raise ValueError("Snapshot must contain an extraction log.")
    list_trade = snapshot["records"]
    extraction_log = snapshot["extraction_log"]
    print(f"Loaded {len(list_trade)} raw records from {SNAPSHOT_PATH}.")
else:
    for i, year in enumerate(YEARS):
        print(f"Downloading {year} ({i + 1}/{len(YEARS)})")
        extracted_at = datetime.now(timezone.utc).isoformat()
        records = extract_year(year)

        # This query should return one aggregate record per year and chapter.
        if len(records) != len(CHAPTER_CODES):
            raise ValueError(
                f"Expected {len(CHAPTER_CODES)} chapter records for {year}, received {len(re
                "Check availability and query scope before continuing."
            )

        list_trade.extend(records)
        extraction_log.append({
            "year_requested": year,
            "records_received": len(records),
            "extracted_at_utc": extracted_at,
            "endpoint": API_URL,
            "query": {**base_params, "period": year}
        })

        if i < len(YEARS) - 1:
            time.sleep(REQUEST_DELAY)

    print(f"Extraction finished: {len(list_trade)} raw records.")
```

```
Downloading 2022 (1/4)
Downloading 2023 (2/4)
Downloading 2024 (3/4)
Downloading 2025 (4/4)
Extraction finished: 384 raw records.
```

3.3 Inspect and validate the raw extraction

First inspect one full record, as in the course template. Then check the year/chapter combinations, reporting dimensions, classification edition, trade values, and extraction-log counts. These are extraction checks, not the final MongoDB cleaning step.

We intentionally keep missing measurements and quality flags unchanged. A zero quantity with an unavailable unit is not automatically a meaningful measured zero. World and commodity totals must not later be summed together with their component records.

```
# Inspect a raw trade document, not the secret-bearing request headers.
pprint(list_trade[0])
```


```
{'aggrLevel': 2,
 'altQty': 0.0,
 'altQtyUnitAbbr': 'N/A',
 'altQtyUnitCode': -1,
 'cifvalue': None,
 'classificationCode': 'H6',
 'classificationSearchCode': 'HS',
 'cmdCode': '01',
 'cmdDesc': 'Animals; live',
 'customsCode': 'C00',
 'customsDesc': 'TOTAL CPC',
 'flowCode': 'X',
 'flowDesc': 'Export',
 'fobvalue': 21097891.879,
 'freqCode': 'A',
 'grossWgt': 0.0,
 'isAggregate': True,
 'isAltQtyEstimated': False,
 'isGrossWgtEstimated': False,
 'isLeaf': False,
 'isNetWgtEstimated': True,
 'isOriginalClassification': True,
 'isQtyEstimated': False,
 'isReported': False,
 'legacyEstimationFlag': 4,
 'mosCode': '0',
 'motCode': 0,
 'motDesc': 'TOTAL MOT',
 'netWgt': 0.0,
 'partner2Code': 0,
 'partner2Desc': 'World',
 'partner2ISO': 'W00',
 'partnerCode': 0,
 'partnerDesc': 'World',
 'partnerISO': 'W00',
 'period': '2022',
 'primaryValue': 21097891.879,
 'qty': 0.0,
 'qtyUnitAbbr': 'N/A',
 'qtyUnitCode': -1,
 'refMonth': 52,
 'refPeriodId': 20220101,
 'refYear': 2022,
 'reporterCode': 757,
```

```
 'reporterDesc': 'Switzerland',
 'reporterISO': 'CHE',
 'typeCode': 'C'}
```

```
expected_keys = {(year, code) for year in YEARS for code in CHAPTER_CODES}
actual_keys = [(str(record.get("period")), record.get("cmdCode")) for record in list_trade]
if len(actual_keys) != len(set(actual_keys)):
    raise ValueError("Duplicate year/chapter records found.")
if set(actual_keys) != expected_keys:
    raise ValueError("Missing or unexpected year/chapter records; check extraction completer

if sorted(entry.get("year_requested") for entry in extraction_log) != sorted(YEARS):
    raise ValueError("Extraction log does not cover each requested year exactly once.")
if any(entry.get("records_received") != len(CHAPTER_CODES) for entry in extraction_log):
    raise ValueError("Extraction-log counts do not match the expected chapters.")

expected_fields = {
    "reporterCode": base_params["reporterCode"],
    "partnerCode": base_params["partnerCode"],
    "partner2Code": base_params["partner2Code"],
    "typeCode": "C",
    "freqCode": "A",
    "aggrLevel": 2,
    "flowCode": base_params["flowCode"],
    "customsCode": base_params["customsCode"],
    "motCode": base_params["motCode"]
}

for record in list_trade:
    for field, expected in expected_fields.items():
        if record.get(field) != expected:
            raise ValueError(f"Unexpected value for {field}; review the response.")
    value = record.get("primaryValue")
    if isinstance(value, bool) or not isinstance(value, (int, float)):
        raise ValueError("A record is missing a numeric primary trade value.")
    if not math.isfinite(value) or value < 0:
        raise ValueError("A record has a non-finite or negative trade value.")
    if record.get("classificationCode") != EXPECTED_EDITION:
        raise ValueError("Unexpected HS edition; resolve comparability before analysis.")

print(f"Validated {len(list_trade)} Swiss export records across {len(YEARS)} years.")
print(f"Chapters per year: {len(CHAPTER_CODES)}; HS edition: {EXPECTED_EDITION}.")
```

```
Validated 384 Swiss export records across 4 years.
Chapters per year: 96; HS edition: H6.
```

3.4 Save the validated raw-data snapshot

In API mode, this cell saves raw records, the query scope, and original download timestamps together in a dated JSON file. It uses a new filename and exclusive creation rather than

overwriting an existing snapshot. It never stores API headers, keys, or database credentials. Snapshot mode leaves the supplied file unchanged.

The professor can use this file with Option B above to repeat the later processing on the same input data. Include the selected snapshot in the submission only after checking ILIAS/course upload guidance and the source's terms; keep it separate from secret files.

UN Comtrade is not an unrestricted redistribution license. Its re-dissemination guidance describes exceptions for limited publications, internal use, and specified free analytical/data-sharing uses. Cite UN Comtrade and check the applicable terms before distributing a larger dataset or publishing raw snapshots on GitHub.

```
if DATA_SOURCE == "api":
    if len(list_trade) != len(YEARS) * len(CHAPTER_CODES) or len(extraction_log) != len(YEAR
        raise ValueError("Complete extraction and validation before saving the dataset.")

    saved_at = datetime.now(timezone.utc)
    snapshot_data = {
        "schema_version": 1,
        "source": "UN Comtrade",
        "source_url": "https://comtradeplus.un.org/ApiDocs",
        "saved_at_utc": saved_at.isoformat(),
        "query_scope": query_scope,
        "extraction_log": extraction_log,
        "records": list_trade
    }
    snapshot_folder = Path("data")
    snapshot_folder.mkdir(exist_ok=True)
    snapshot_name = f"trade_snapshot_{saved_at:%Y%m%dT%H%M%S%fZ}.json"
    saved_snapshot_path = snapshot_folder / snapshot_name

    with saved_snapshot_path.open("x", encoding="utf-8") as file:
        json.dump(snapshot_data, file, ensure_ascii=False, indent=2, allow_nan=False)

    print(f"Saved {len(list_trade)} raw records to {saved_snapshot_path}.")
    print("For offline reproduction, select snapshot mode and use this path.")
else:
    print("Using the supplied snapshot; no new snapshot was written.")
```

```
Saved 384 raw records to data/trade_snapshot_20261015T182736439029Z.json.
For offline reproduction, select snapshot mode and use this path.
```

```
# Pandas is only for inspection here; list_trade remains the raw input for MongoDB.
preview_columns = [
    "period", "reporterDesc", "partnerDesc", "cmdCode",
    "classificationCode", "primaryValue", "netWgt", "isNetWgtEstimated"
]
df_preview = pd.DataFrame(list_trade)
df_preview.reindex(columns=preview_columns).head(10)
```

3.5 Readable trade-data CSV

Export all extracted rows and columns, not only the ten-row preview. CSV is a convenient flat-table copy for inspection. Keep the JSON snapshot for reproduction and load `list_trade` directly into MongoDB later: a CSV round-trip can lose types, null distinctions, nested structures, and leading zeros in commodity codes.

```
Path("data").mkdir(exist_ok=True)
trade_csv = Path("data") / f"swiss_exports_{datetime.now(timezone.utc):%Y%m%dT%H%M%S%fZ}.csv
df_preview.to_csv(trade_csv, index=False, mode="x")
print(f"All {len(df_preview)} trade records saved to {trade_csv}")
```

```
All 2 trade records saved to data/swiss_exports_20261015T1827423368822.csv
```

```
# The log contains query settings and timestamps, but no API key.
pd.DataFrame(extraction_log)[
    ["year_requested", "records_received", "extracted_at_utc"]
]
```

	year_requested	records_received	extracted_at_utc
0	2022	96	2026-10-15T18:20:30.222184+00:00
1	2023	96	2026-10-15T18:20:39.457310+00:00
2	2024	96	2026-10-15T18:20:46.539782+00:00
3	2025	96	2026-10-15T18:20:52.909484+00:00

3.6 Extraction interpretation and next steps

A complete extraction should contain **384 records: 96 chapters × 4 years**, with Switzerland as reporter, World as partner, and H6 as the classification edition. The validations are specific to this declared scope, not proof of completeness for every possible query.

Next, preserve the snapshot and load these raw records into MongoDB. Use a unique key based on the observation dimensions so repeated imports do not create duplicates. Model reporter, partner, commodity, measures, and quality flags as related embedded fields after loading. A separate tree collection is not needed for chapter-level analysis.

Perform the main cleaning and analytical aggregations inside MongoDB. Calculate rankings, chapter shares, changes, and top-five concentration there rather than in Pandas. Missing or invalid weight measures should not prevent value-based analysis, but must be documented. Percentage growth needs an explicit rule for zero baselines. The final submission still needs diagrams, conclusions, learnings, and a checked PDF.

4 Sources and method references

- UN Comtrade API documentation: endpoints, query parameters, and authentication.
- UN Comtrade API guide: public preview versus authenticated access.
- Subscription plans: check current free-tier limits before a larger extraction.
- Commodity classification and conversion issues: HS revisions and comparability.

- Trade valuation: interpretation of primary trade values and import/export valuation differences.
- Course project template: the `requests` → JSON → list → load workflow.
- Local course notes: PyMongo insertion, document modeling, and MongoDB transformations for subsequent stages.

Trade values are in current US dollars. They are not inflation-adjusted. Download timestamps are recorded when the extraction cell runs; source data may be revised.

5 PDF export and final submission checklist

The first Markdown cell contains Quarto YAML settings. Quarto supplies the title, table of contents, and section numbering, so headings do not contain manually typed numbers. Code remains visible to document the method. Rendering is configured **not to execute Python cells**: the PDF uses the notebook's saved outputs.

From a terminal opened in this notebook's folder, render with:

```
quarto render "UN_commodity.ipynb" --to pdf --no-execute
```

Before the final export:

- Review the project title/subtitle and add your author details to the YAML header.
- Complete the analysis, conclusions, learnings, and source references; remove or rewrite this working checklist for the submitted report.
- Include at least two images, one being the class diagram. Use relative paths, readable labels, and captions.
- Keep result tables narrow, with clear units and sensible precision. A Pandas table that looks good in Jupyter may need adjustment for LaTeX PDF output.
- Restart the kernel, run all cells successfully in Jupyter, and save. Running the notebook performs the configured requests and, once implemented, database operations; inspect the rerun policy first.
- Check code and saved outputs for credentials, tracebacks, excessive logs, and unnecessary raw dumps. Hide verbose cells selectively for the PDF without removing reproducible code from the notebook.
- Render and inspect every page for clipped code or tables, broken images, duplicated numbering, and awkward page breaks. The PDF has no Swiss data results until the updated extraction cells have run and their outputs have been saved.
- Package the final PDF, executed notebook, and image files in a ZIP. If the upload rules accept additional files, include the selected dated snapshot, `.env.example`, dependencies, and rerun instructions. Exclude `.env`, unused snapshots, and temporary render artifacts. Check relative image and snapshot paths after extracting the ZIP.

Quarto and TinyTeX were available during the initial setup check. If rendering later becomes impractical, browser Print → Save as PDF is an assignment-approved fallback.